



TEMA 8 - Responsive Design

Responsive Design

Diseño adaptable a todos los dispositivos

Qué aprenderás:

- Adaptar layouts a distintos tamaños de pantalla
- Usar media queries correctamente
- Trabajar con unidades flexibles
- Aplicar Mobile First

¿Qué es el Responsive Design?

Es la técnica que permite que una web:



Se vea bien en móvil



Se adapte a tablet



Funcione en escritorio

Sin crear varias webs distintas.

Mobile First vs Desktop First

Mobile First (RECOMENDADO)

Se diseña primero para móvil y luego se escala.

```
/* estilos base → móvil */  
  
@media (min-width: 768px) {  
  /* tablet */  
}
```

Desktop First

@media (max-width: 768px)

Viewport

Etiqueta obligatoria en HTML:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Permite:

- Que el diseño responda al ancho real del dispositivo

Unidades relativas

Más usadas en responsive:

- %
- em
- rem
- vw
- vh
- fr (grid)

vw y vh

```
width: 50vw;
```

```
height: 100vh;
```

Muy útiles para:

- Hero sections
- Layouts completos

Ejercicio 1

Crear una sección que:

- Ocupe 100vh
- Centre el contenido vertical y horizontalmente
- Use unidades flexibles para el texto

Media Queries

```
@media (min-width: 768px) {  
  
}
```

Breakpoints típicos

(No obligatorios)

- 480px → móvil
- 768px → tablet
- 1024px → portátil
- 1280px → desktop

Mobile First real

```
.card {  
  width: 100%;  
}  
  
@media (min-width: 768px) {  
  .card {  
    width: 50%;  
  }  
}
```

Ejercicio 2

Crea 1 Layout responsive:



1 columna en móvil



2 columnas en tablet



4 columnas en desktop

Flexbox y responsive

Flexbox adaptable

```
flex-wrap: wrap;
```

Cambio de dirección:

```
flex-direction: column;
```

en desktop:

```
flex-direction: row;
```

Ejercicio 3

Crear una navbar que:



En móvil:

- Logo arriba
- Menú en columna



En escritorio:

- Logo izquierda
- Menú horizontal

Grid y responsive

Grid auto-fit y minmax()

Esto es MUY profesional:

```
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
```

3 auto-fit

“Crea **tantas columnas como quepan en el contenedor**”

1 grid-template-columns

Define **cuántas columnas hay y cuánto mide cada una.**

2 repeat()

Sirve para repetir columnas sin escribirlas a mano.

4 minmax(250px, 1fr)

Esto define **cuánto puede medir cada columna.**

Ejercicio 4

Crear una galería que:

- Se adapte sola al ancho
- No use media queries
- Mantenga proporciones

Imágenes Responsive

Imágenes fluidas

```
img {  
  max-width: 100%;  
  height: auto;  
}
```



object-fit

`object-fit: cover;`
`object-fit: contain;`



fill



cover



contain



scale-down



none

contain+none
100% 100%

Ejercicio 5

Crear una card que:



En móvil:

- Imagen arriba
- Texto abajo



En desktop:

- Imagen izquierda
- Texto derecha

Tipografía responsive

```
font-size: clamp(1rem, 2.5vw, 2rem);
```

Permite texto fluido.



¿Qué hace **clamp()**?

```
clamp(MIN, VALOR IDEAL, MAX)
```

Significa:



“Usa el valor ideal,
pero **nunca bajas del mínimo ni subas del máximo**”

Cómo elegir los valores

Regla práctica para clase:

Para títulos

```
font-size: clamp(1.8rem, 4vw, 3rem);
```

Para subtítulos

```
font-size: clamp(1.4rem, 3vw, 2.2rem);
```

Para texto normal

```
font-size: clamp(1rem, 1.2vw, 1.2rem);
```

Ejercicio 6

Crear:

- h1
- h2
- p

que escalen con el viewport.

BUENAS PRÁCTICAS Y ERRORES

Reglas de oro

- ✓ Mobile First
- ✓ Evitar valores fijos en px
- ✓ Usar grid/flex
- ✓ Probar en dispositivos reales

Errores comunes

- ✗ Ocultar contenido importante en móvil
- ✗ Botones pequeños
- ✗ Textos ilegibles
- ✗ Layouts que rompen